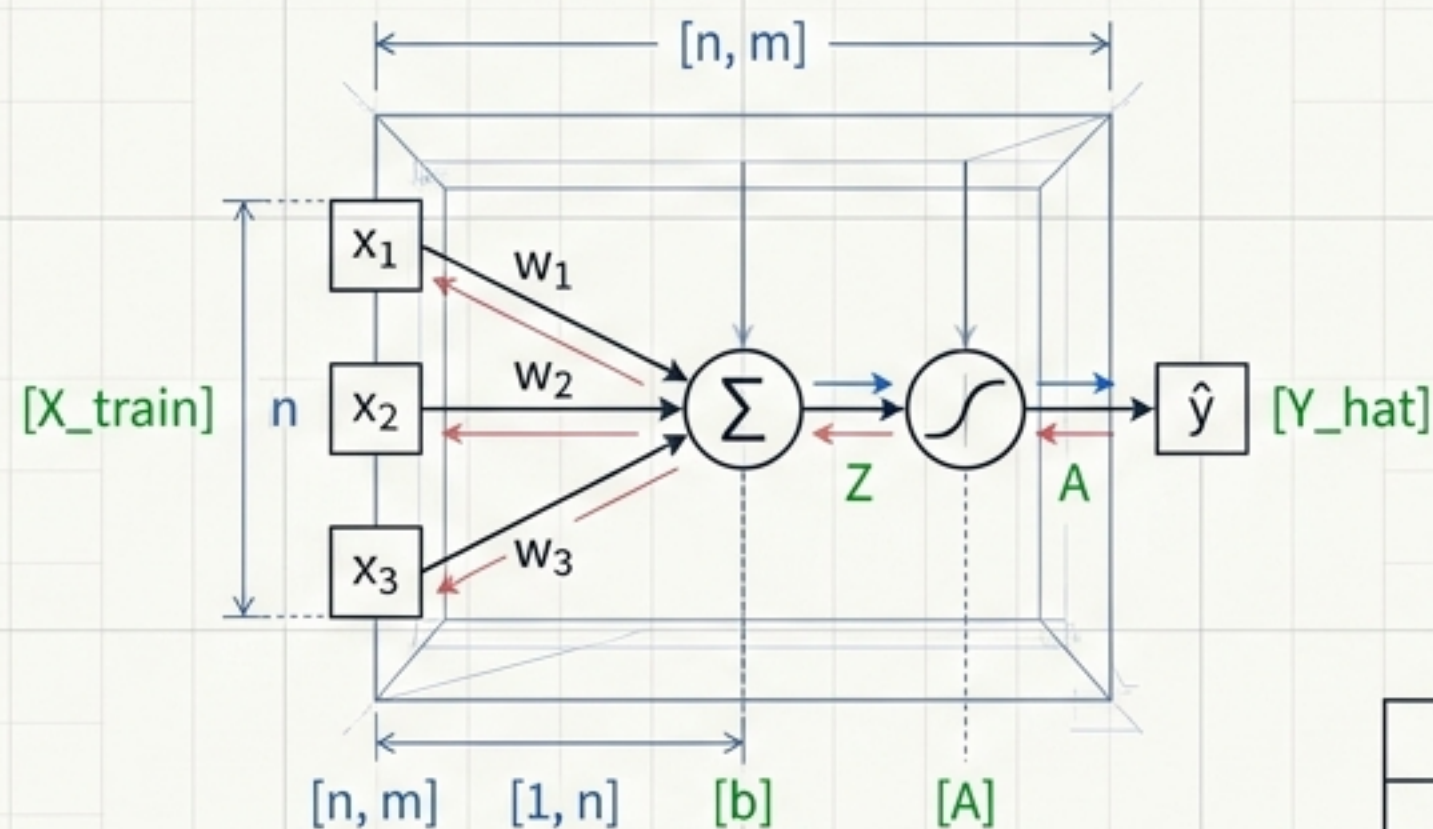


신경망 프로그래밍의 기초: 이진 분류와 로지스틱 회귀

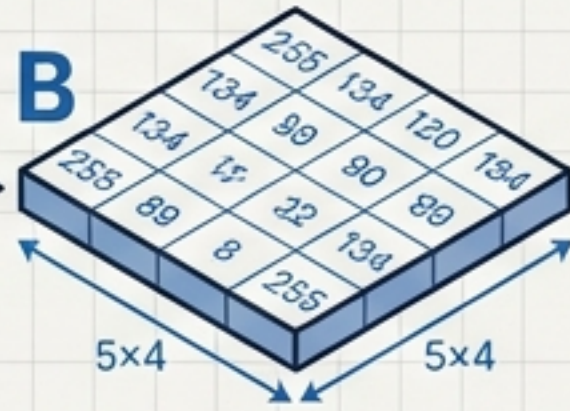
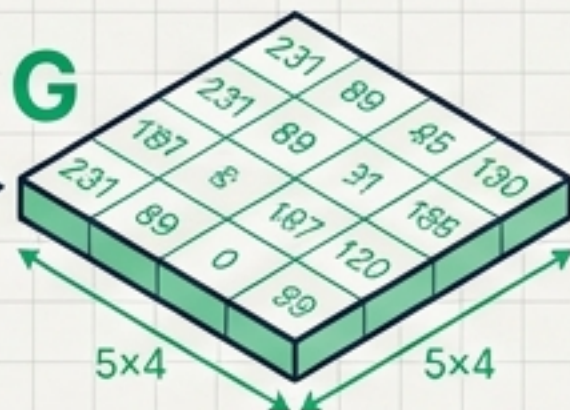
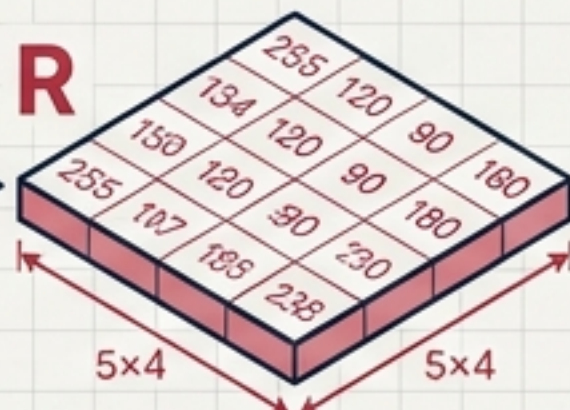
데이터 구조화부터 경사 하강법과 계산 그래프의 수학적 해부까지



Legend	
	정방향 패스 (Forward): 예측값 계산
	역전파 (Backward): 기울기(도함수) 계산
	(Code & Dims): 파이썬 변수명 및 행렬 크기

문제 정의: 입력된 이미지 x 가
고양이(1)인지 아닌지(0) 분류하는
이진 분류(Binary Classification).

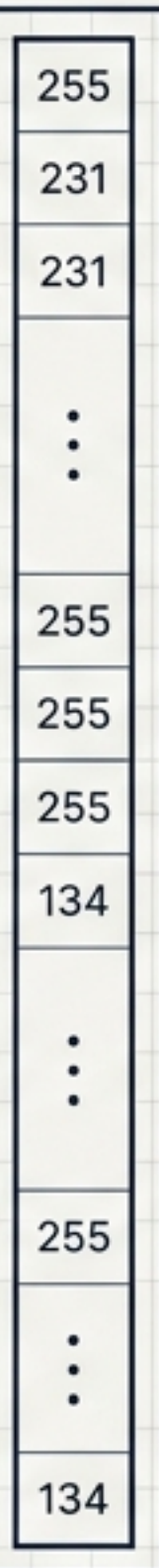
INPUT IMAGE
(64x64x3)



UNROLLING
PROCESS

DIMENSION
REDUCTION
FLOW

FEATURE VECTOR x

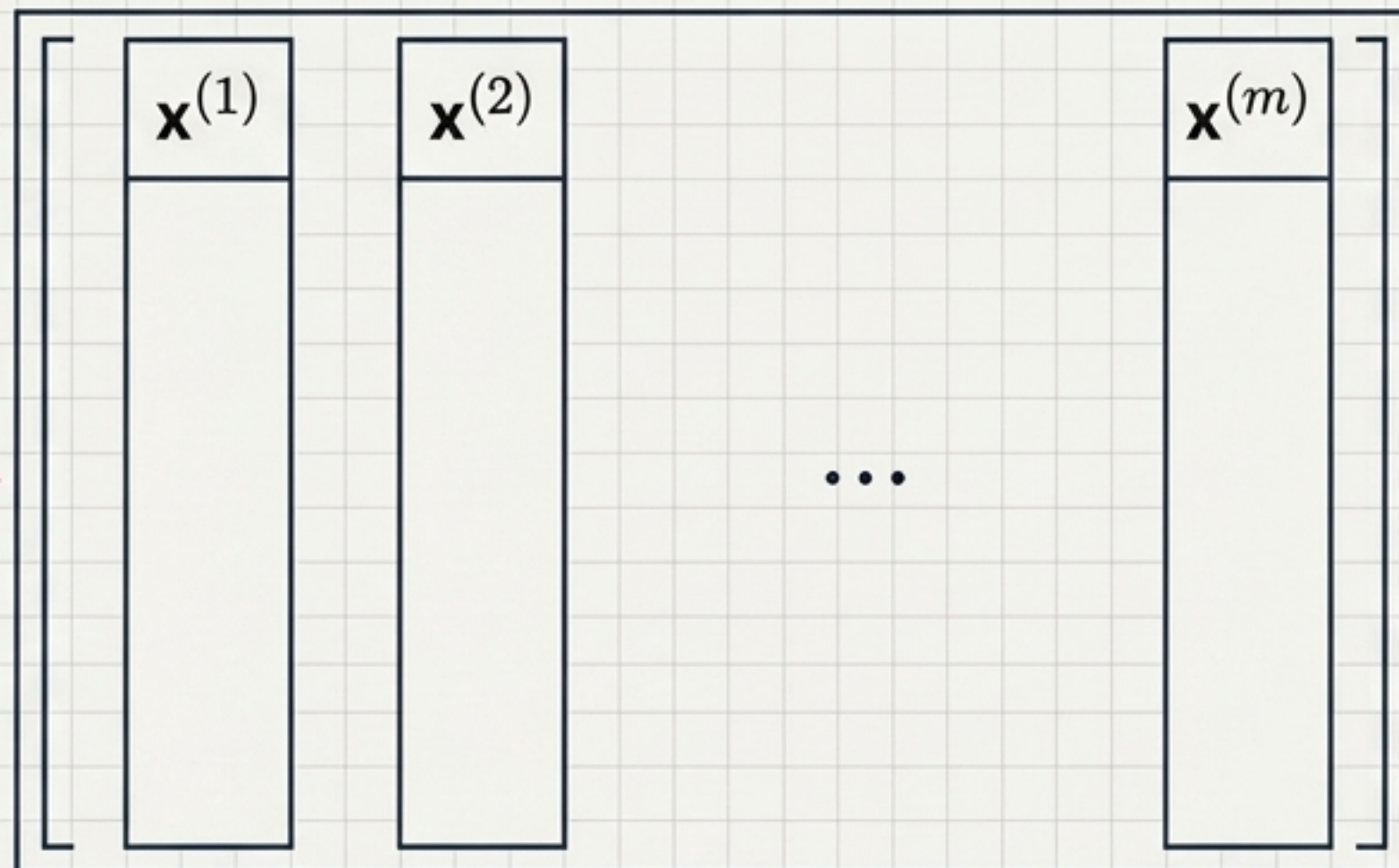


입력 특성 벡터의 차원
 $n_x = 12,288$

특성 벡터 (Feature
Vector) x : 64 x 64 x 3
개의 픽셀 강도 값을 단일
열 벡터로 나열.

훈련 세트 $\mathbf{X}$ (Training Set)

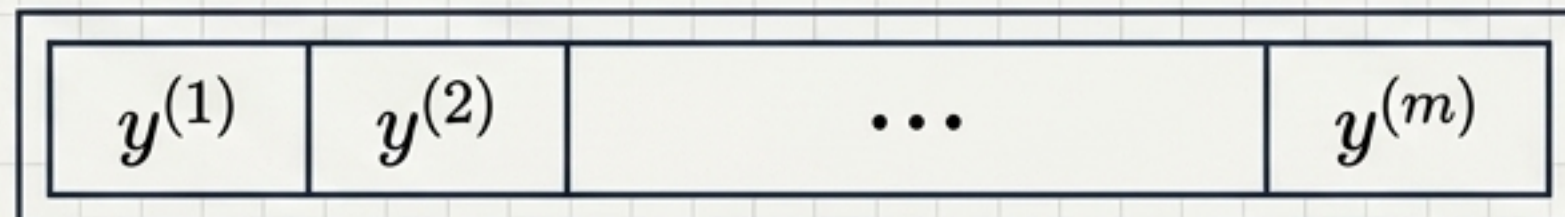
$\mathbf{X}$: 훈련 샘플을 열(Column)
방향으로 쌓아 올린 행렬
(신경망 구현에 최적화된 형태)



$\mathbf{X.shape} = (n_x, m)$

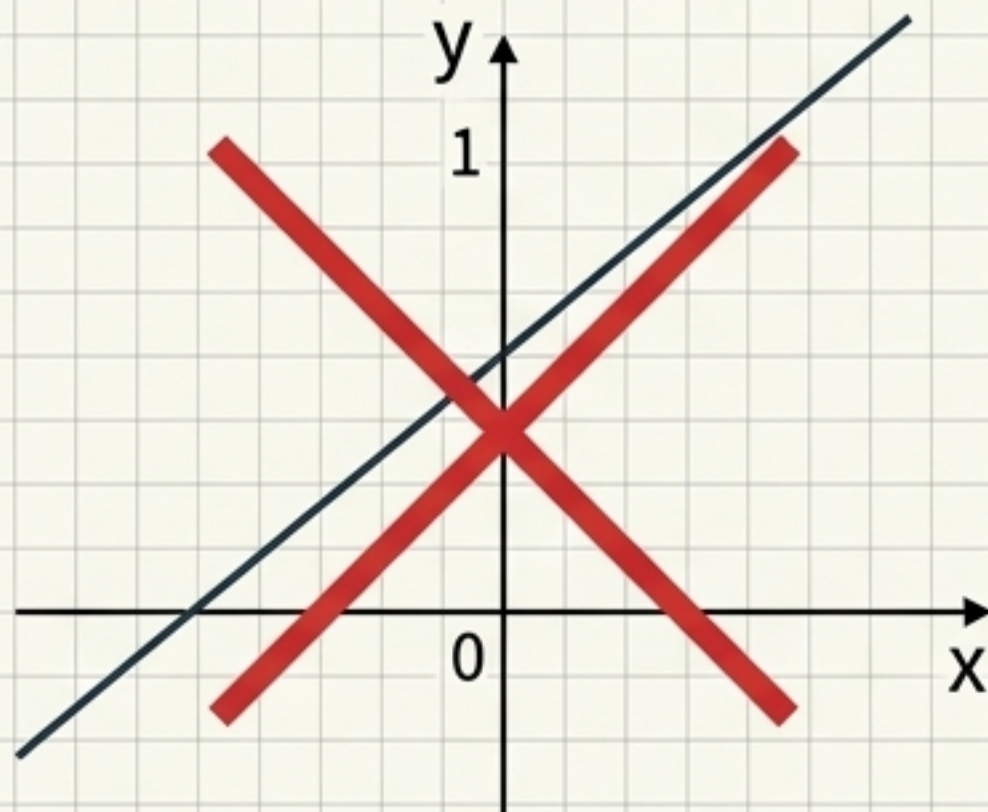
출력 레이블 $\mathbf{Y}$ (Output Labels)

$\mathbf{Y}$: 각 샘플에 대응하는
0 또는 1의 정답 세트



$\mathbf{Y.shape} = (1, m)$

선형 회귀의 한계

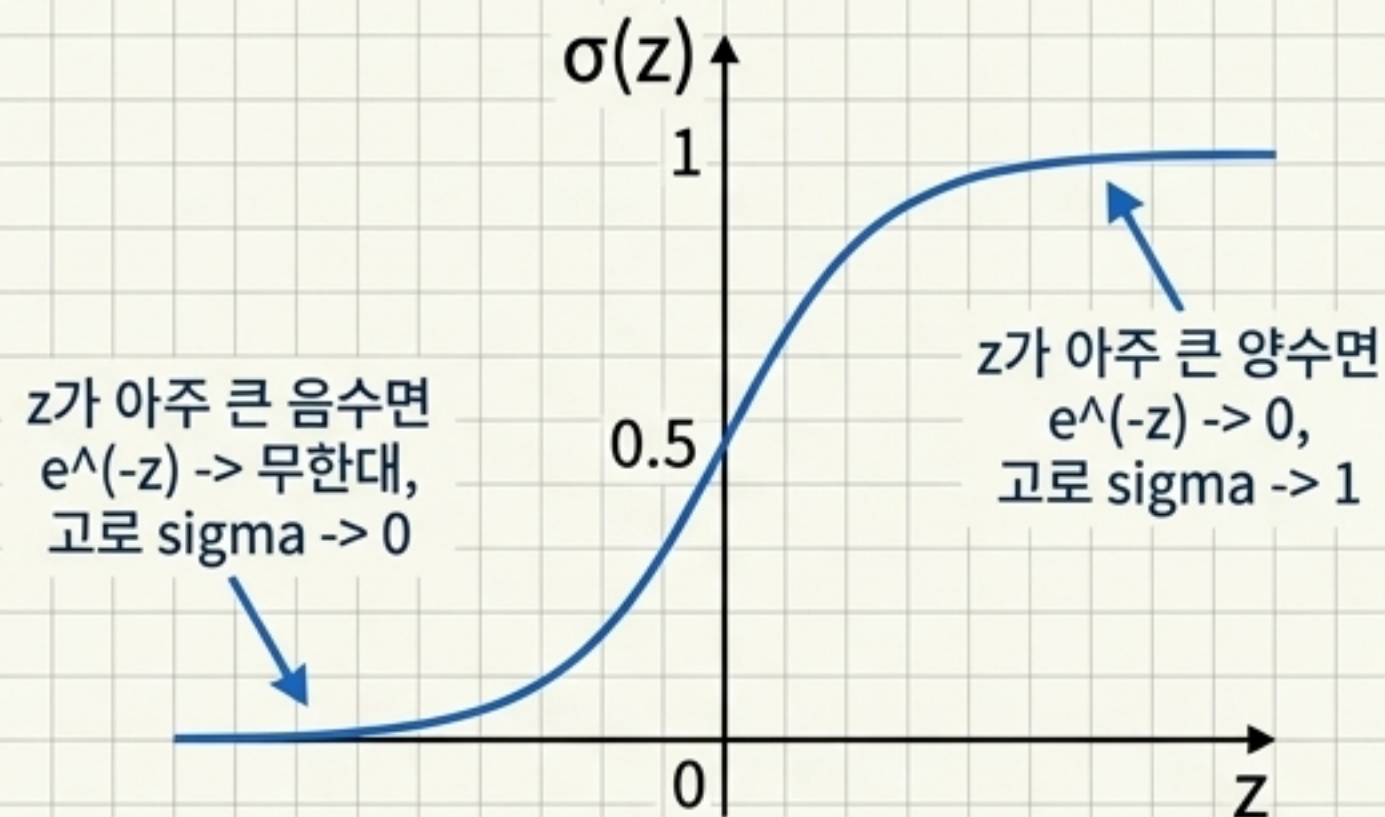


$$\text{수식: } \hat{y} = w^T x + b$$

문제점: 확률 범위를 벗어난 음수나 1 이상의 큰 값이 출력될 수 있음.

출력값 $\hat{y}$ 은 항상 $0 \leq \hat{y} \leq 1$ 인 '확률'이 됩니다.

로지스틱 회귀 모델: 시그모이드

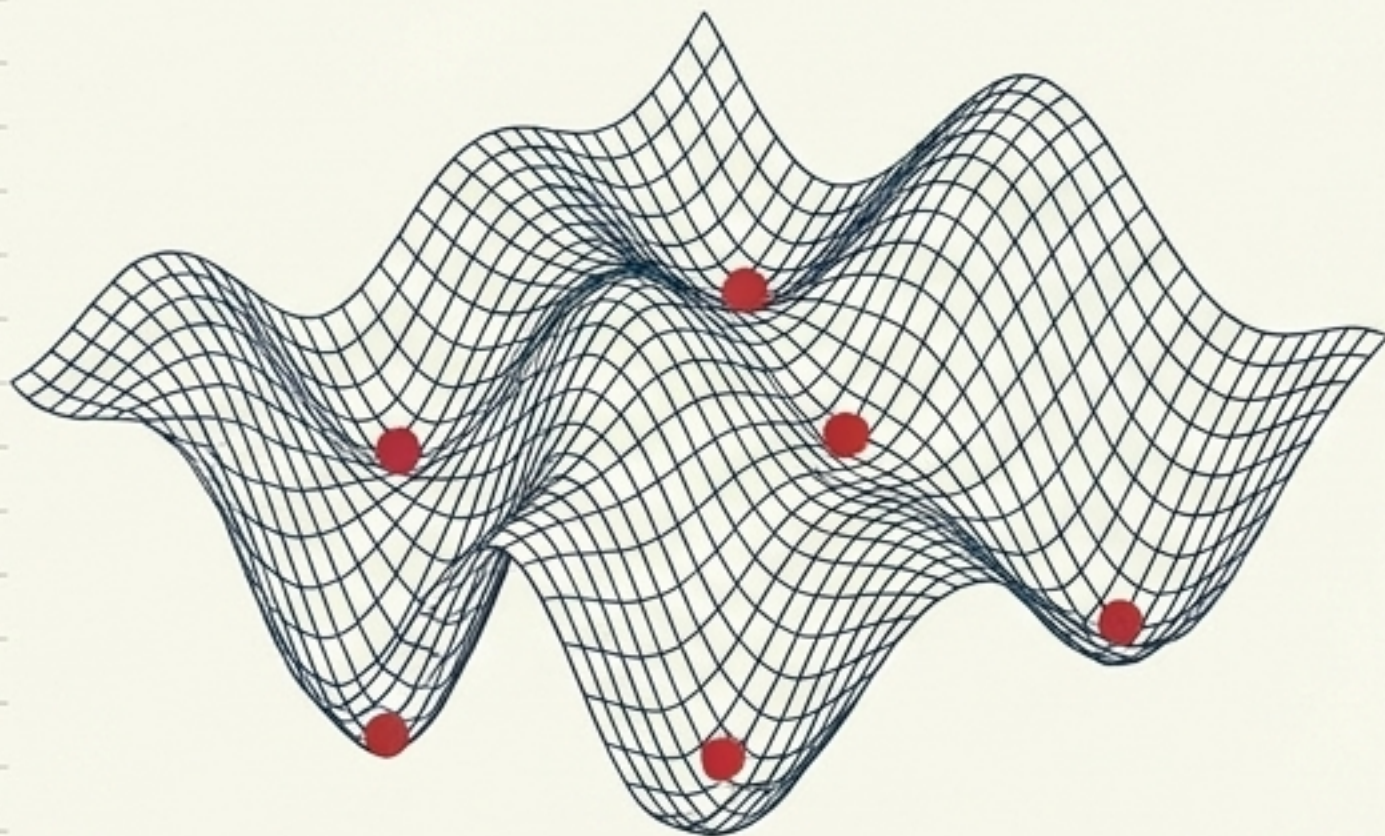


$$\text{모델: } \hat{y} = \sigma(w^T x + b)$$

$$\text{공식: } \sigma(z) = \frac{1}{1 + e^{-z}}$$

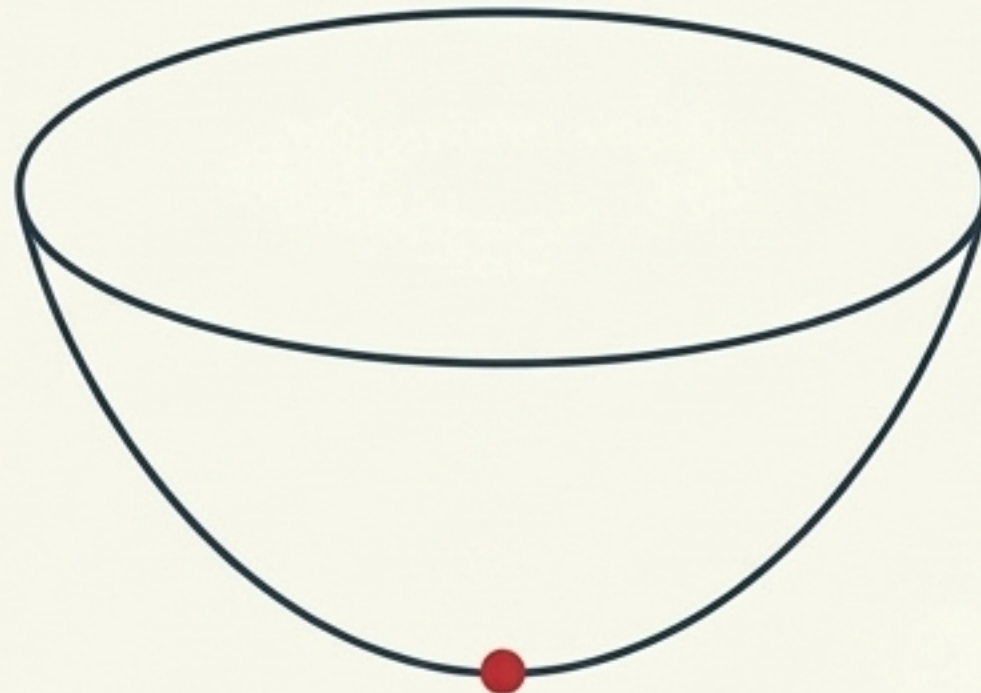
오차 측정: 손실 함수 (Loss Function)

오차 제곱합 (MSE)



로지스틱 회귀에 적용 시
함수가 볼록(Non-convex)하지 않아
경사 하강법이 전역 최적값을 찾지 못함.

로지스틱 손실 함수 (Log-Loss)



$$\text{수식: } L(\hat{y}, y) = -(y \log(\hat{y}) + (1-y) \log(1-\hat{y}))$$

$y = 1$ 일 때

$$L = -\log(\hat{y})$$

오차를 줄이려면 $\hat{y}$ 이
1에 가까워져야 함

$y = 0$ 일 때

$$L = -\log(1-\hat{y})$$

오차를 줄이려면 $\hat{y}$ 이
0에 가까워져야 함

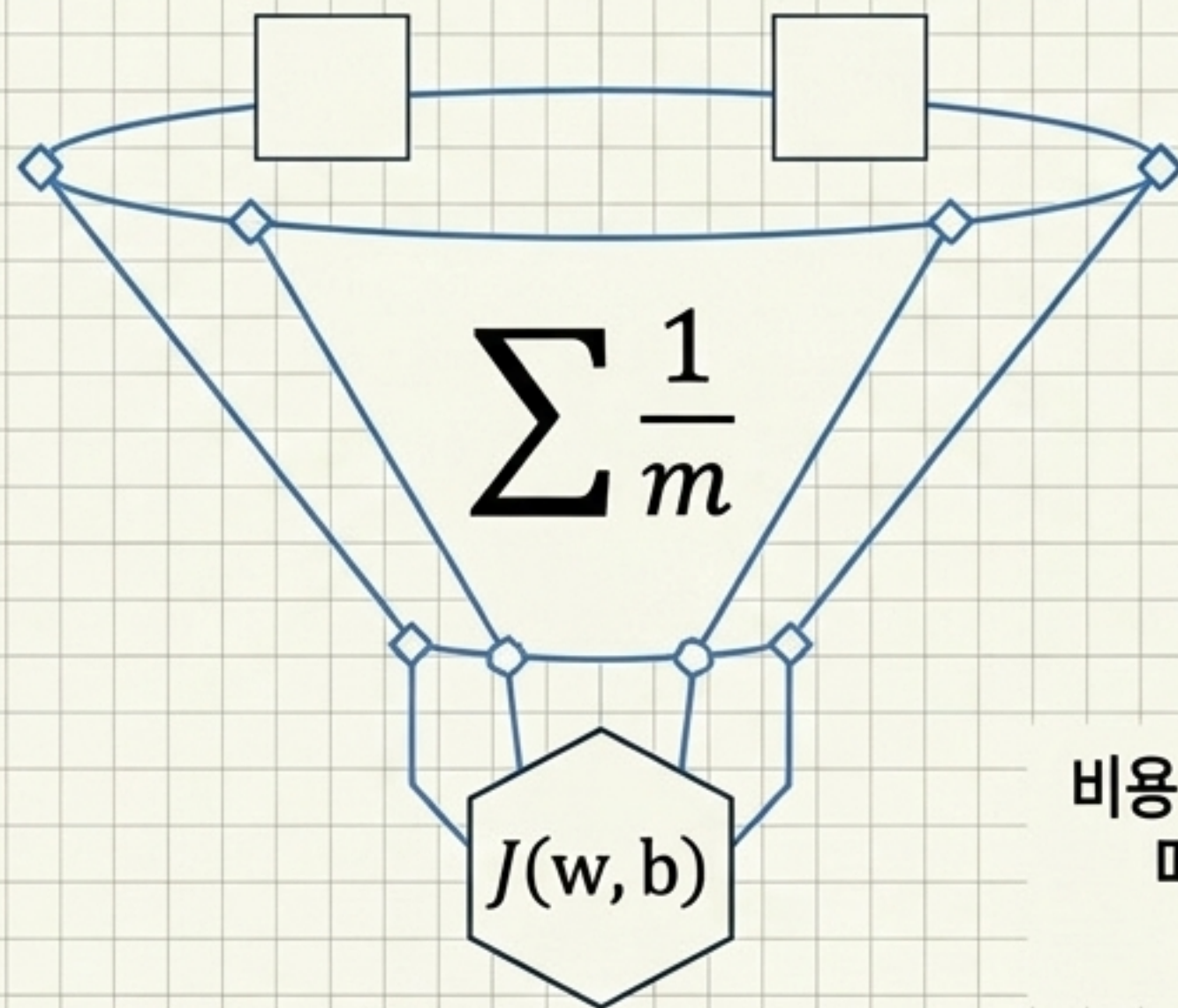
$$L(y_{\text{hat}}^{(1)}, y^{(1)})$$

$$L(y_{\text{hat}}^{(2)}, y^{(2)})$$

...

$$L(y_{\text{hat}}^{(m)}, y^{(m)})$$

손실 함수 L: 단일 훈련
샘플(1개)에 대한 오차 측정.

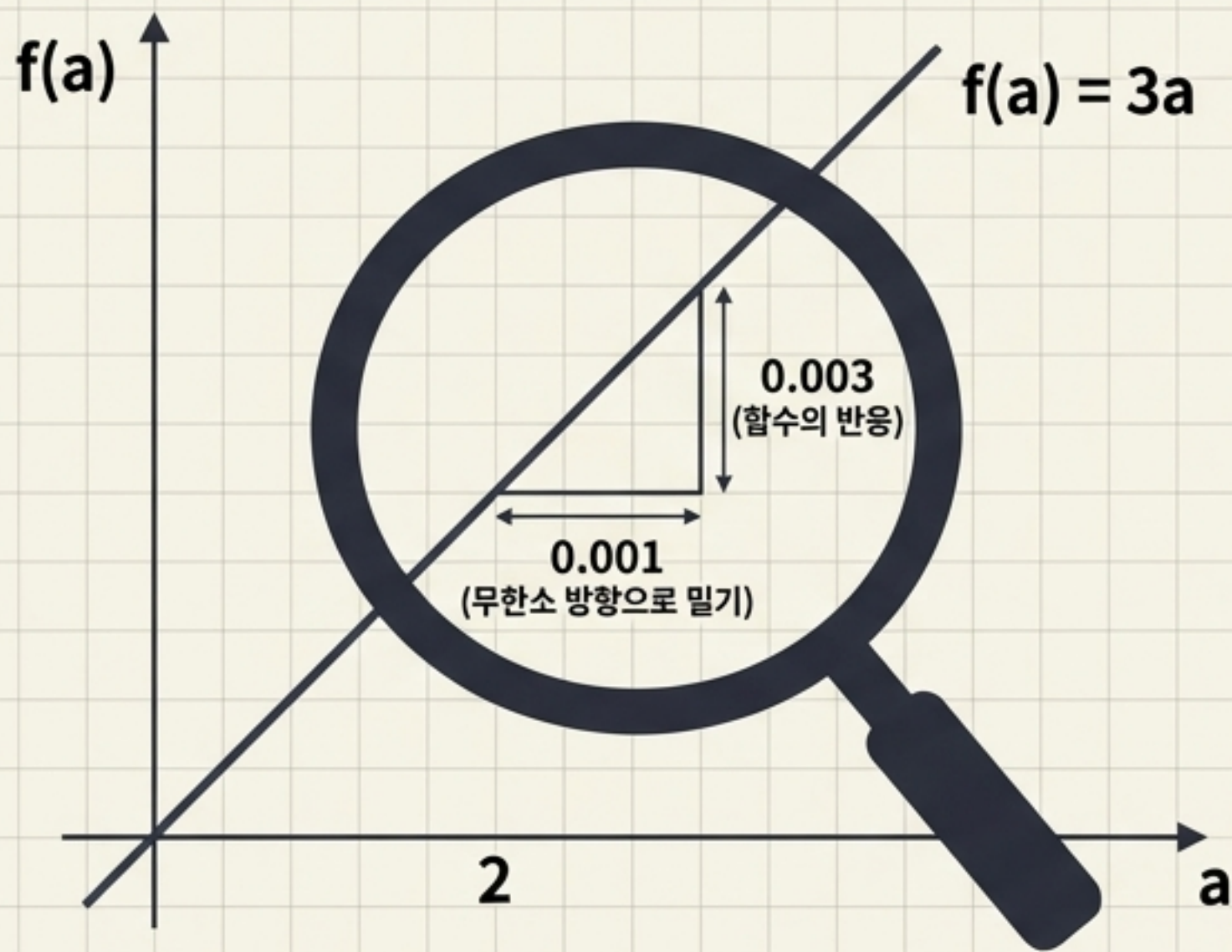


비용 함수 J: 훈련 세트 전체(m개)에 대한
매개변수 w, b의 예측 적합도 측정
(손실들의 평균).

$$J(w, b) = -\frac{1}{m} \sum [y^{(i)} \log(\mathbf{y_hat}^{(i)}) + (1 - y^{(i)}) \log(1 - \mathbf{y_hat}^{(i)})]$$

최종 목표: 비용 함수 $J(w, b)$ 의 값을 최소화하는 매개변수 w와 b를 찾는 것.

도함수 = 기울기



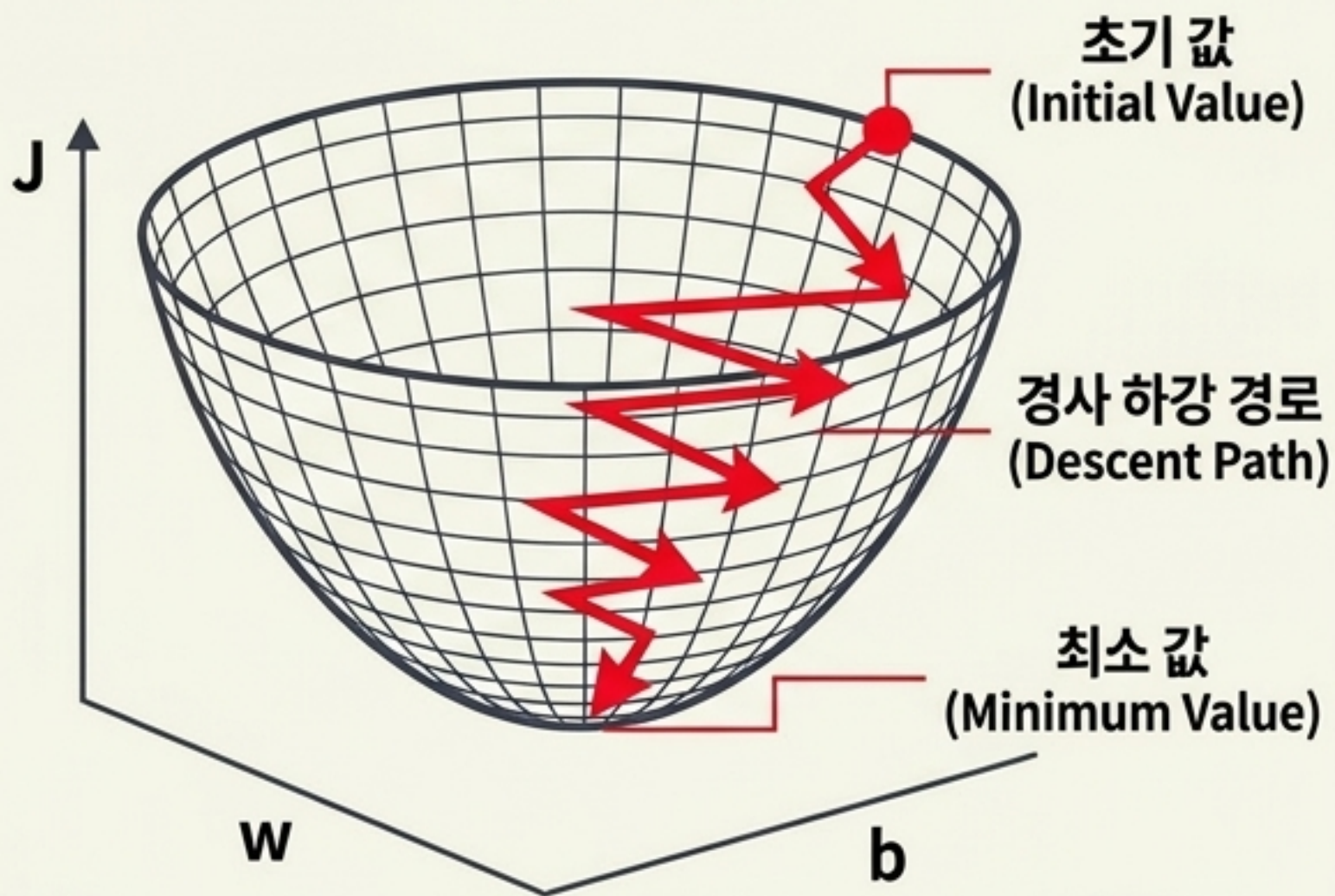
변수 a 를 무한소(직관적으로는 0.001)만큼 밀었을 때,
함수 $f(a)$ 가 반응하여 변화하는 배수 (비율 1:3)

기울기가 변하는 함수들 (미분 공식 체화)

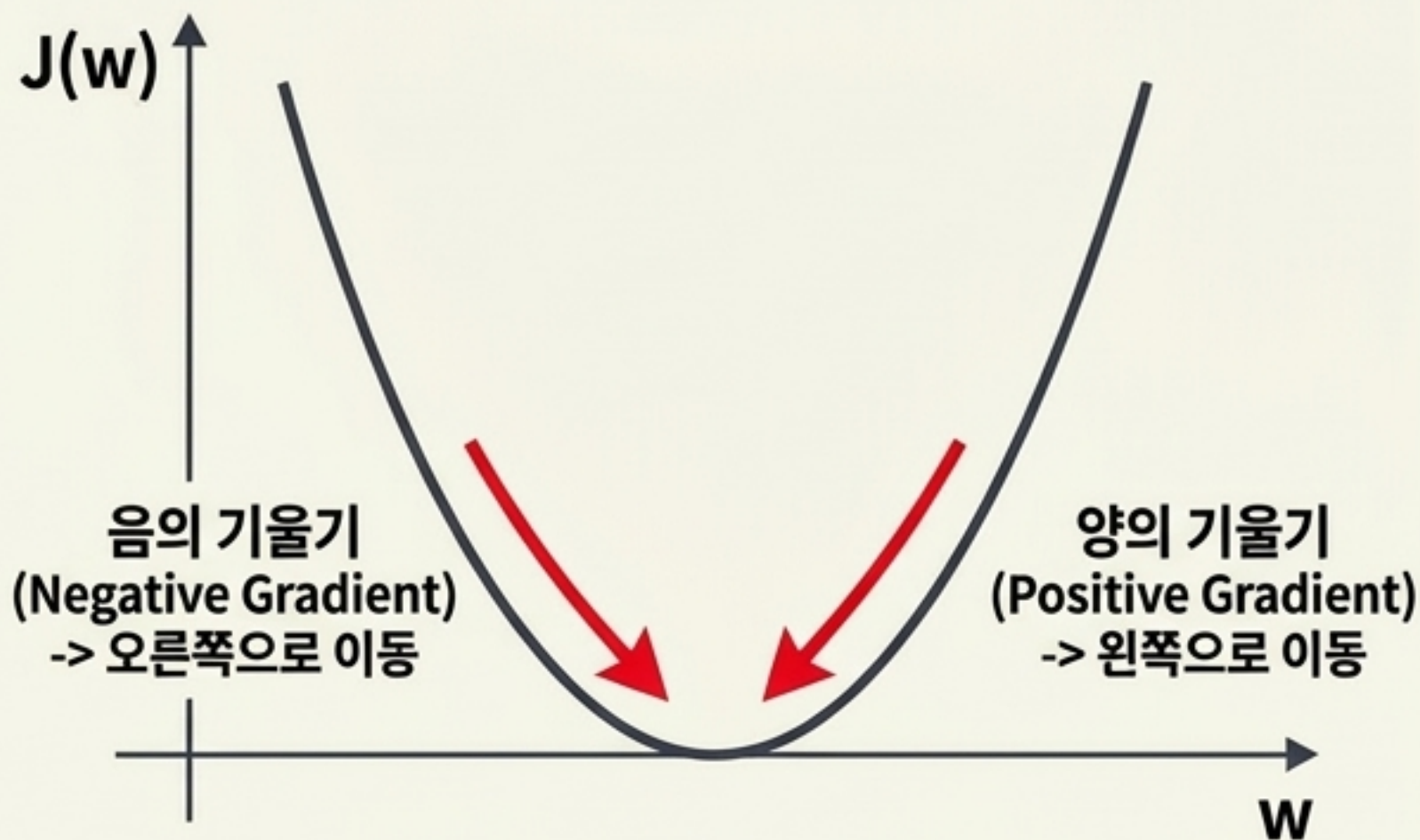
$f(a) = a^2$	기울기 (도함수): $2a$	$a=2$ 일 때 0.001 증가시키면 값은 $4 * 0.001$ = 0.004 증가
$f(a) = \log(a)$	기울기 (도함수): $1/a$	$a=2$ 일 때 기울기는 0.5

경사 하강법 (Gradient Descent)

3D View



1D Cross-Section

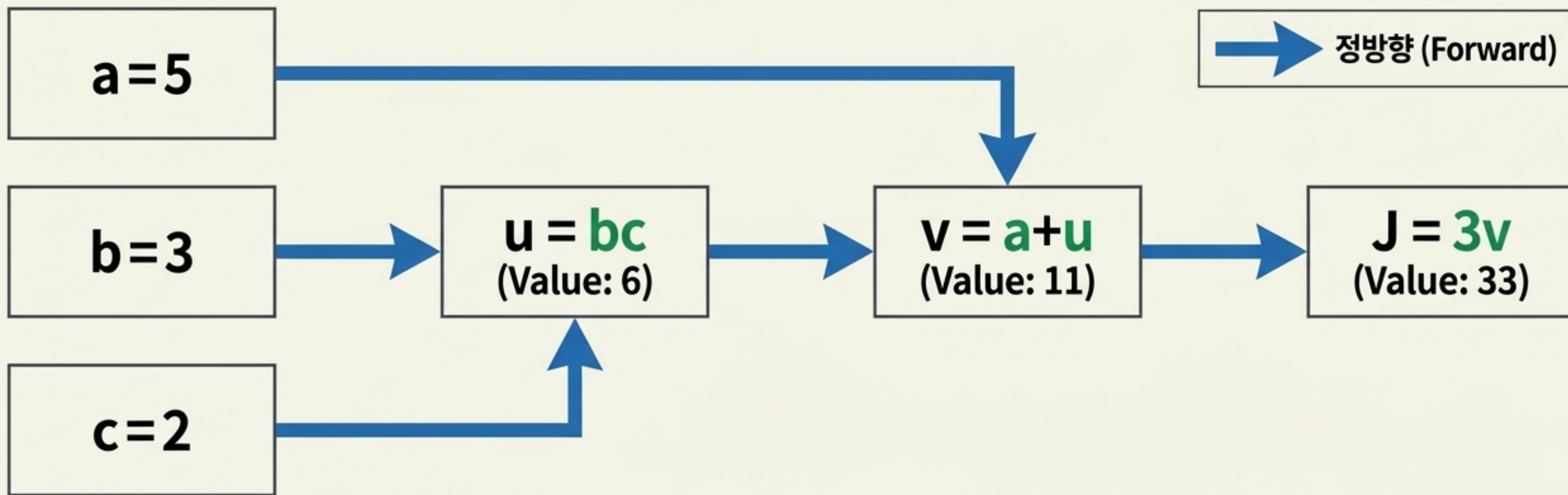


공식 해부 (Formula Anatomy)

$$\underline{w} := w - \underline{\alpha} * \underline{dw}$$

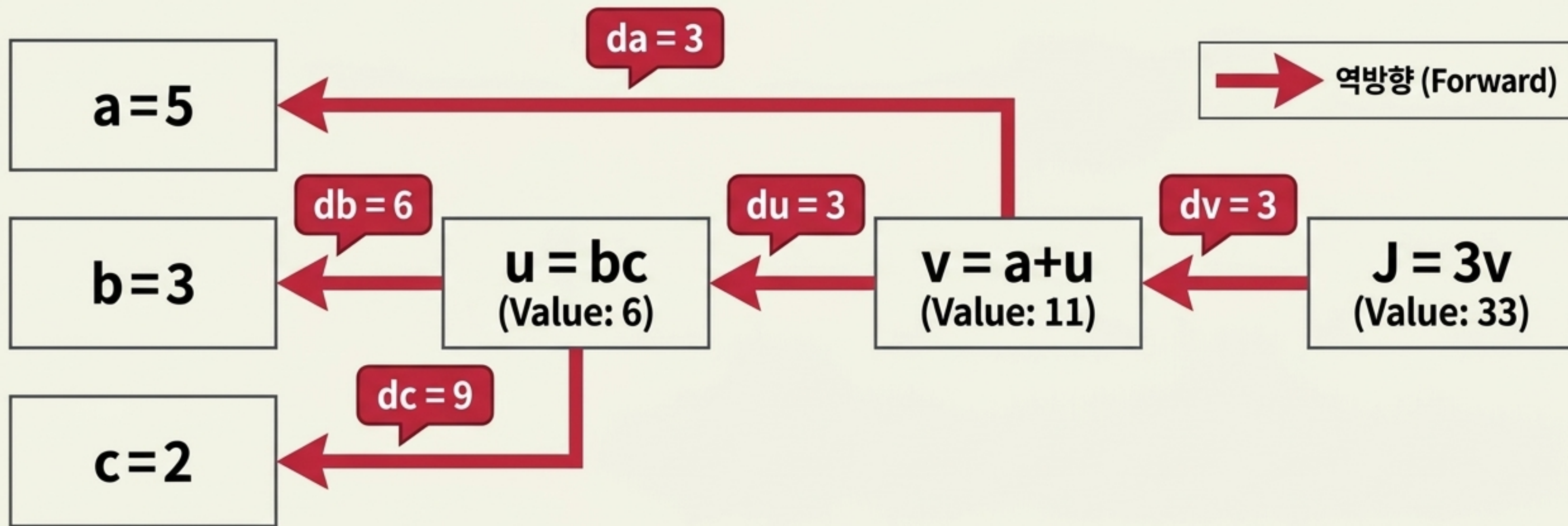
값을 갱신 (Update) 학습률: 한 번에 내려갈 보폭의 크기 수학적 편미분 기호 dJ/dw 의 파이썬 코드 변수명. J 가 w 방향으로 얼마나 기울었는지(가장 가파른 내리막 방향)를 나타냄.

계산 그래프의 원리 (Computation Graph) - 정방향 패스



정방향 전파 (Forward Pass): 입력 변수들로부터 시작해 중간 변수들을 거쳐 최종 출력 값(비용 함수 J)을 계산하는 과정.

역전파와 연쇄 법칙 (Backpropagation & Chain Rule)

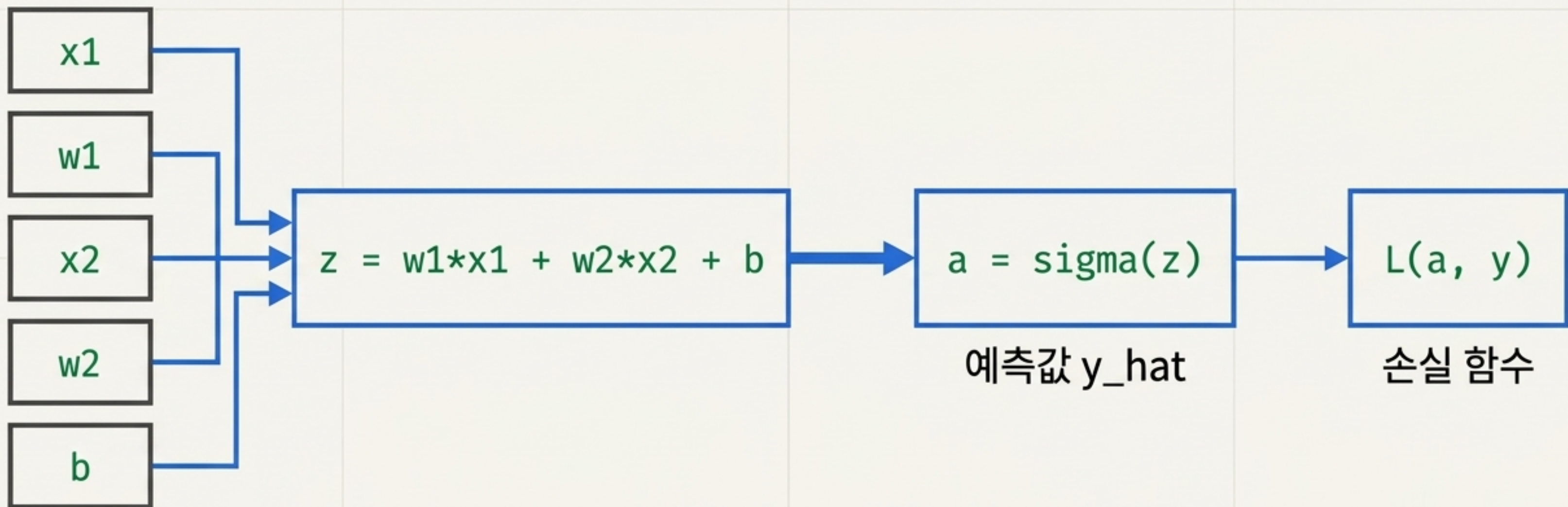


역방향 전파 (Backward Pass)
최종 출력값의 변화량을 알기 위해 우측에서 좌측으로 도함수를 연쇄적으로 곱해나감.

연쇄 법칙 (Chain Rule)
$$\frac{dJ}{da} = \frac{dJ}{dv} * \frac{dv}{da} \rightarrow 3 * 1 = 3$$

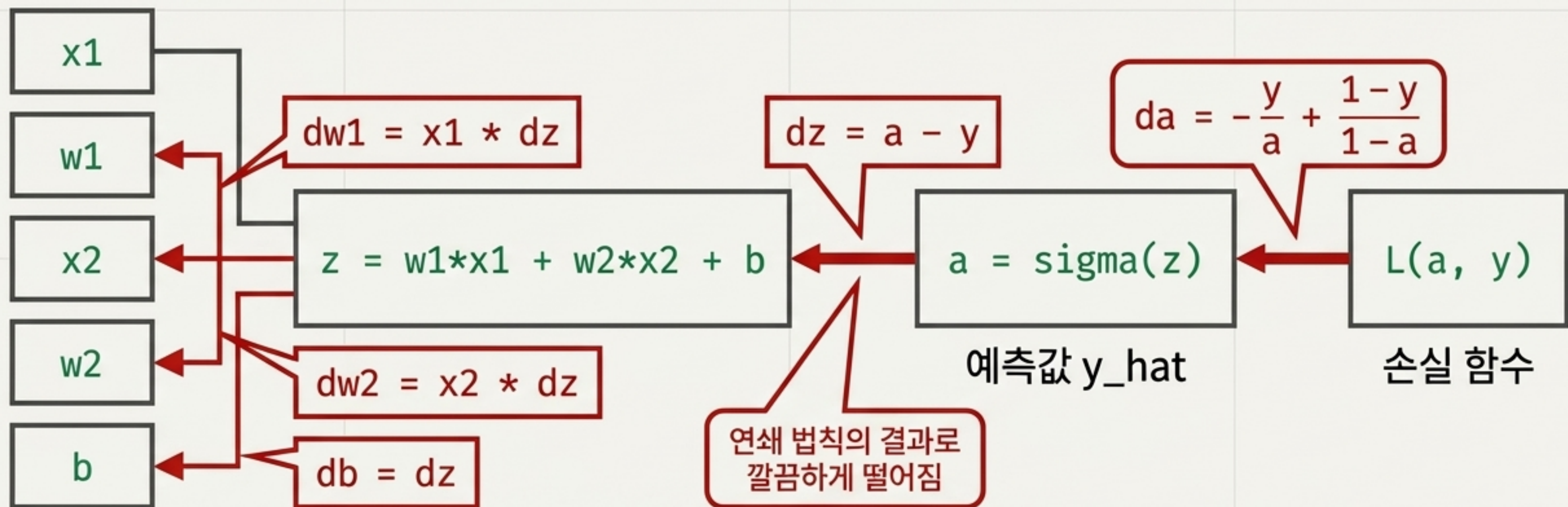
코드 명명 규칙
최종 출력값(J)을 특정 변수(var)로 미분한 도함수 $dJ/dvar$ 는 코드에서 간단히 'dvar'로 표기. (예: dv, da)

통합 I: 로지스틱 회귀의 정방향 전파



단일 훈련 샘플에 대한 로지스틱 회귀의 정방향 전파입니다.
매개변수(w , b)로부터 비용 함수의 손실(L)을 도출해 내는 흐름.

통합 II: 로지스틱 회귀의 역전파



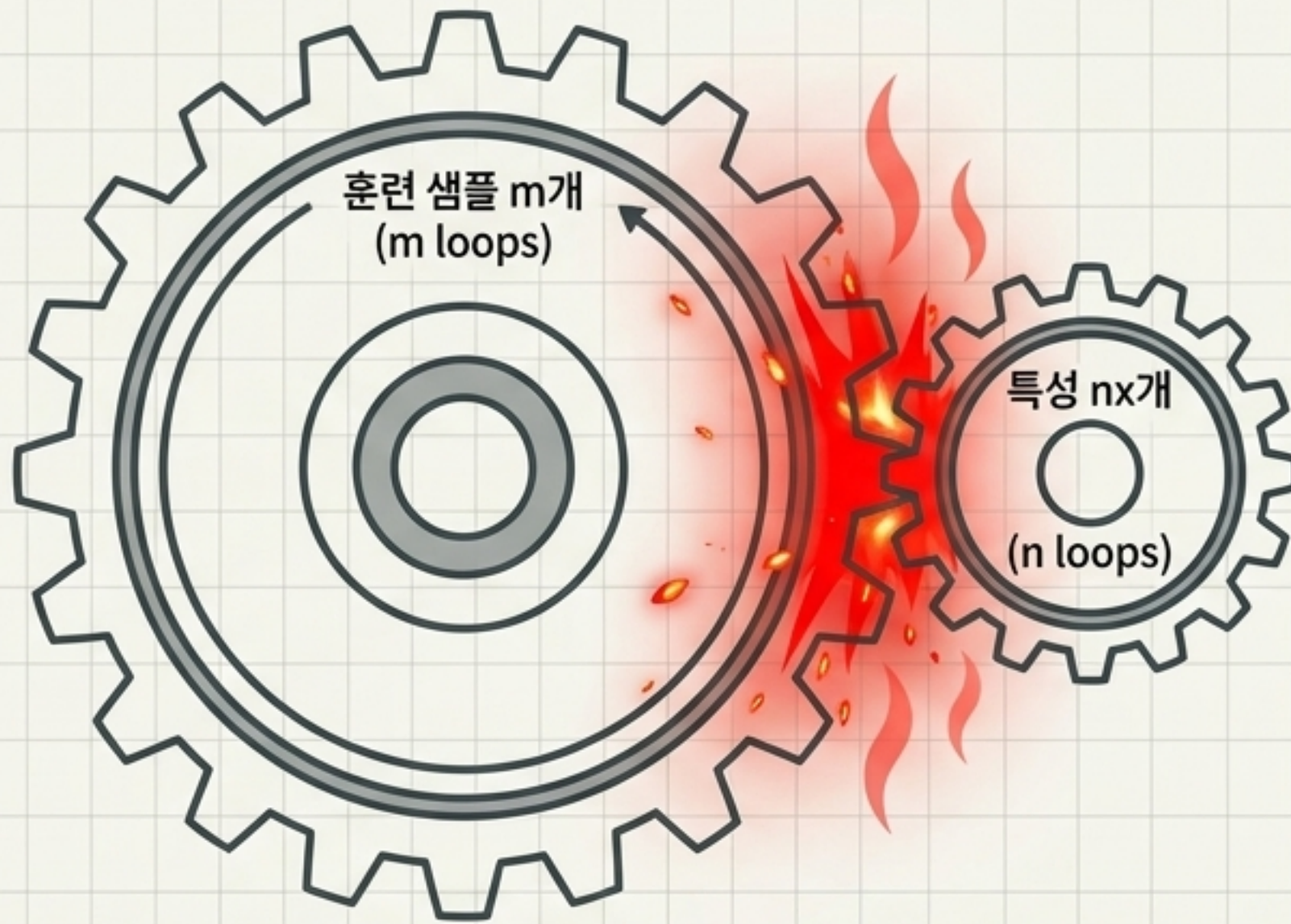
역전파 핵심 공식:

- 미적분을 모르더라도 $dL/dz = a - y$ 로 떨어지는 이 결과를 코드에 구현하면 됩니다.
- 이 값들(dw_1, dw_2, db)이 매개변수 업데이트($w := w - \alpha * dw$)의 보폭을 결정합니다.

m개의 훈련 샘플을 위한 구현 (Scaling to m)

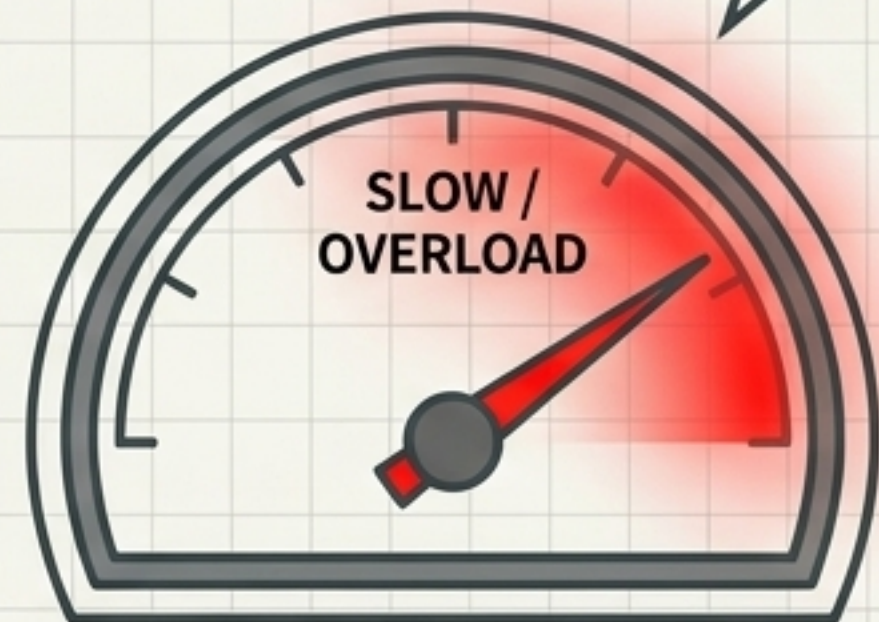
```
J = 0, dw1 = 0, dw2 = 0, db = 0 (초기화)
For i = 1 to m:
    z(i) = wTx(i) + b
    a(i) = sigma(z(i))
    J += -[y(i)log(a(i)) + (1-y(i))log(1-a(i))] (손실 누적)
    dz(i) = a(i) - y(i)
    dw1 += x1(i) * dz(i) (기울기 누적)
    dw2 += x2(i) * dz(i)
    db += dz(i)
J /= m
dw1 /= m, dw2 /= m, db /= m (평균 도출)
w1 := w1 - alpha * dw1
w2 := w2 - alpha * dw2
b := b - alpha * db
```

명시적 for 문의 한계 (The Problem with For-Loops)



병목의 근원:

훈련 샘플 m 에 대한 루프와,
수만 개의 특성 개수 nx 에 대한 루프까지
이중 For 문을 요구함.



거대한 데이터셋(딥러닝 시대)에서
파이썬의 명시적 For 문(Explicit For-loops)은
치명적인 병목 현상을 유발합니다.



해결책: 벡터화 (Vectorization).
코드를 명시적 For 문 없이 행렬
연산으로 한 번에 처리하여 속도를
비약적으로 높이는 기법.

The Blueprint Cheat Sheet: 마스터 요약본

데이터 차원 & 변수 규칙 (Dimensionality)

입력 행렬 X : shape (n_x, m)

정답 행렬 Y : shape $(1, m)$

dvar 규칙: 최종 변수에 대한
편미분값

예: $dw = dJ/dw$

정방향 전파 (Forward Pass)

$$z = w^T x + b$$

$$a = \text{sigma}(z) = 1 / (1 + e^{-z})$$

$$J = -1/m \sum [y \log(a) + (1-y)\log(1-a)]$$

역전파 & 최적화 (Backward & Optimize)

$$dz = a - y$$

$$dw = 1/m \sum (x * dz)$$

$$db = 1/m \sum dz$$

$$w := w - \text{alpha} * dw$$

$$b := b - \text{alpha} * db$$

혼자 코딩할 때 벽에 붙여놓고 볼 수 있는 핵심 수식 매핑 테이블.